Testing Data Analysis Workflows in R

A practical guide to testthat, assertr, and reproducibility testing for research pipelines

Ronald 'Ryy' G. Thomas

2025-07-25

Table of contents

Figure 1: A research workstation with test results displayed alongside data analysis output

Testing data analysis workflows transforms fragile scripts into reliable research infrastructure.

1 Introduction

I did not really know how to systematically test a data analysis pipeline until I applied software engineering practices from testthat and assertr to my own research workflow. Data analysis testing presents challenges that differ fundamentally from traditional software testing: you must validate data quality, ensure computational reproducibility, and verify that analytical results are both correct and meaningful.

For years I relied on informal validation – checking that results "looked reasonable" and trusting that my code was correct because it ran without errors. These approaches failed to scale. When

a collaborator's data arrived with unexpected column types, my pipeline broke silently. When a package update changed default behaviour, my results shifted without warning.

This post presents the testing taxonomy I developed for data analysis workflows: unit tests for individual functions, data validation tests for quality assurance, integration tests for full pipelines, and reproducibility tests for deterministic results. All examples use the Palmer Penguins dataset and are immediately applicable to your own research code.

1.1 Motivations

- My research pipeline broke after a data pull introduced unexpected missing values in a column I assumed was complete.
- A collaborator could not reproduce my results because a package update changed the default random seed handling.
- I was writing the same validation checks repeatedly across projects and wanted a systematic, reusable approach.
- The gap between software engineering testing practices and data science workflows seemed unnecessarily wide.
- As Wilson et al. (2017) note, "An important aspect of this is to validate the code using software development practices that prevent errors and software testing methods that can help detect them when they occur."

1.2 Objectives

1. Distinguish between computational reproducibility (the pipeline works) and result correctness (the results are accurate) as separate testing goals.
2. Implement unit tests, data validation tests, integration tests, and reproducibility tests using testthat and assertr.
3. Organise test files following R package conventions and demonstrate how to run them from the terminal.
4. Set up continuous integration with GitHub Actions to automate testing on every push.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

Systematic testing adapts software engineering rigour to the specific needs of data analysis.

2 Prerequisites and Setup

Install the required packages:

```
install.packages(
  c("testthat", "assertr",
    "palmerpenguins", "dplyr",
    "ggplot2", "validate")
)
```

Load libraries:

```
library(testthat)
library(assertr)
library(palmerpenguins)
library(dplyr)
library(ggplot2)
```

Load the sample data:

```
data(penguins)
glimpse(penguins)
```

Background assumed: Familiarity with R and the tidyverse. No prior experience with testing frameworks is required.

3 What is Testing for Data Analysis?

Testing for data analysis is the practice of writing automated checks that verify your analysis pipeline produces correct, reproducible results. Unlike traditional software testing, which focuses on function inputs and outputs, data analysis testing must also address data quality, statistical properties, and computational determinism.

Think of it as quality control for research code. Just as a manufacturing process has inspection points that verify each component meets specifications, a data analysis pipeline has tests that verify each stage – data loading, cleaning, modelling, and reporting – produces expected results.

Testing serves two primary goals. Computational reproducibility means another researcher can use your code and data to obtain identical results. Result correctness means the results generated by the code are accurate and meaningful. Both goals require different types of tests, and both are necessary for trustworthy research.

4 Getting Started

4.1 The Testing Taxonomy

Test Type	What It Checks	When to Use
Unit	Individual functions	Helper functions
Data validation	Data quality	After loading
Integration	Full pipeline	Before finalising
Reproducibility	Same results	Random processes

4.2 Unit Tests

Unit tests verify that individual functions behave correctly with known inputs and expected outputs:

```
test_that("outlier detection works", {
  test_data <- c(1, 2, 3, 100, 4, 5)
  outliers <- detect_outliers(
    test_data, method = "iqr"
  )

  expect_equal(outliers, 100)
  expect_length(outliers, 1)
})
```

4.3 Data Validation Tests

Data validation tests ensure data meets quality requirements before analysis proceeds:

```
test_that("data meets quality standards", {
  data <- read.csv(
    "analysis/data/raw_data/penguins.csv"
  )

  expect_equal(ncol(data), 8)
  expect_true(
    all(
      c("species", "body_mass_g") %in%
        names(data)
    )
  )
  expect_true(
    all(data$body_mass_g > 0, na.rm = TRUE)
  )
```